Absolutely. The best way to learn this architecture is **not** to start with FAST-LIO2, FUEL, ESDF, B-splines, etc. as isolated algorithms. First understand the **information flow through the drone**.

Your whole system can be explained with one sentence:

> **Sense the environment → estimate where the drone is → build a map → decide where to explore → generate a safe trajectory → command the flight controller → move → repeat.**

The survivor pipeline runs alongside this and answers:

> **See a person → determine where that person is in 3D → transform that position into the map → tag the survivor.**

That framing matches the NIDAR mission: autonomous GPS-denied navigation, continuously updated 2D mapping, autonomous survivor detection/localisation, and live display at the GCS. Mission Brief - NIDAR AirMouse

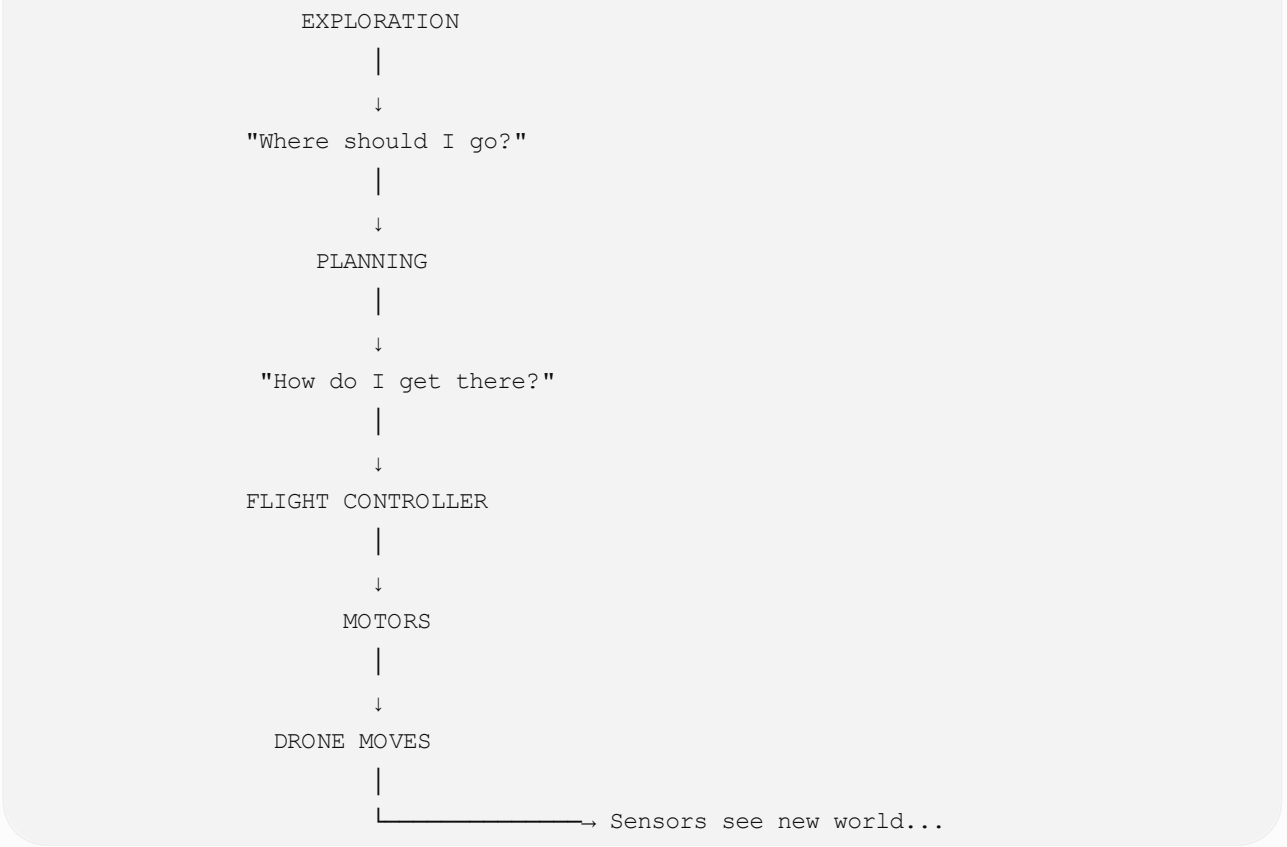
---

1. First understand the drone as a closed loop

Forget the individual software packages temporarily.

Your drone is essentially doing this continuously:

```
                UNKNOWN WORLD
                     |
                     ↓
                  SENSORS
                     |
                     ↓
          "What am I seeing?"
                     |
                     ↓
            STATE ESTIMATION
                     |
                     ↓
             "Where am I?"
                     |
                     ↓
                 MAPPING
                     |
                     ↓
          "What is around me?"
                     |
                     ↓
```



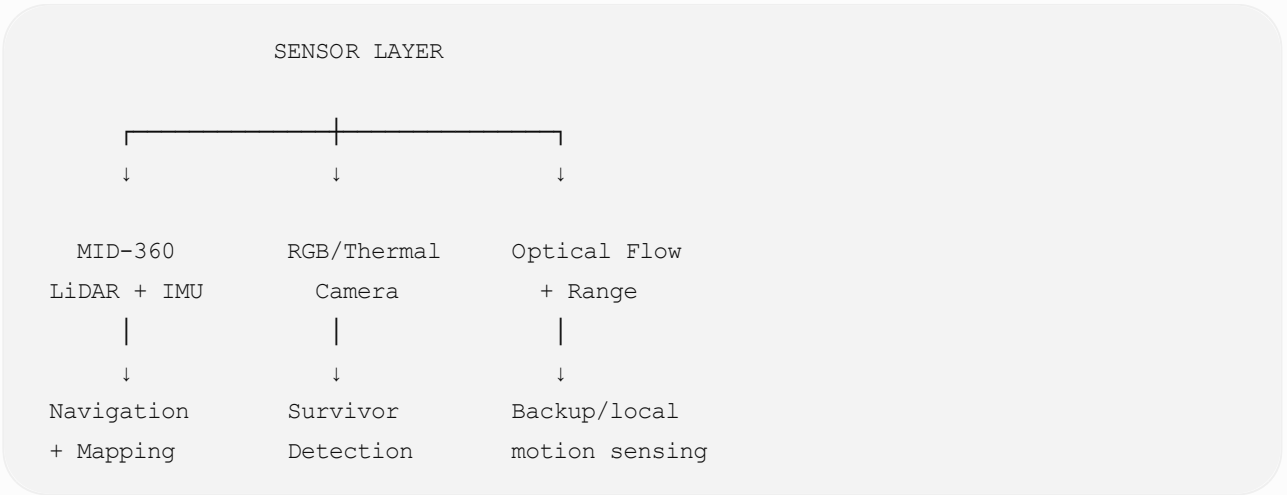
This loop could execute **hundreds of times during one mission**.

That is the fundamental concept I would explain to the team before discussing any individual algorithm.

---

2. SENSOR LAYER — "What does the drone know?"

Your proposed sensor layer has three major branches:



They have **different jobs**.

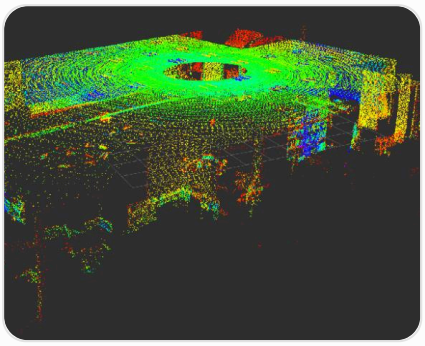
A common mistake is thinking:

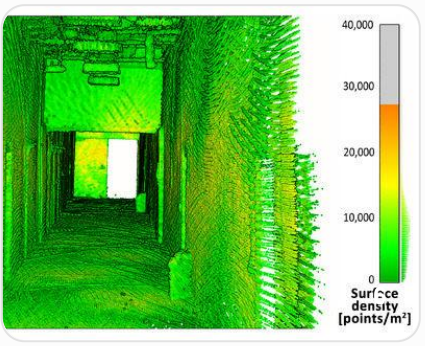
> "All these sensors tell the drone where it is."

Not quite.

---

3. MID-360 — the drone's geometric eyes



The MID-360 continuously emits laser pulses and measures their return.

Conceptually:

```
Laser emitted
     ↓
hits wall
     ↓
laser reflected
     ↓
LiDAR measures return time
     ↓
distance calculated
```

So instead of receiving a normal photograph, it receives thousands of **3D points**.

Imagine the drone sitting here:

```
          WALL

    • • • • • • • • •
    •
    •
    •       DRONE
    •         X
    •
    •
    • • • • • • • • •
```

Every `•` represents a measured 3D location.

A point could mathematically look like:

```
x = 2.34 m
y = -0.82 m
z = 1.17 m
```

Thousands of these become a:

Point cloud

```
MID-360
    |
    ↓
[x,y,z]
[x,y,z]
[x,y,z]
[x,y,z]
[x,y,z]
...
    |
    ↓
POINT CLOUD
```

But there's an important problem.

These coordinates are initially relative to the **LiDAR**, not your global maze map.

The LiDAR knows:

> "There is a wall 2 metres from me."

It does **not inherently know**:

> "I am at coordinate `(6.2, 4.1)` inside the NIDAR arena."

That's where FAST-LIO2 enters.

---

4. Why the LiDAR has an IMU

Suppose the drone rotates.

At time `t1`:

```
      ↑ drone
```

100 ms later:

```
      → drone
```

The LiDAR measurements changed dramatically.

Did the walls move?

No.

The drone moved.

Therefore the system must separate:

```
environment motion
vs
drone motion
```

The IMU measures things such as:

```
Angular velocity
       +
Linear acceleration
```

So:

```
MID-360 LiDAR
      │
      │ geometry
      ↓
   FAST-LIO2
      ↑
      │ motion
      │
LiDAR-associated IMU
```

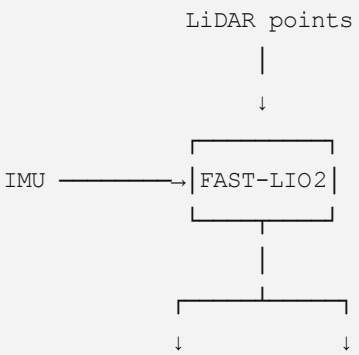
FAST-LIO's central idea is precisely this: tightly fuse LiDAR feature measurements with IMU measurements using an iterated Kalman filter. The paper also uses forward propagation from IMU measurements and backward propagation to compensate for motion distortion within a LiDAR scan. 2010.08196v3

---

5. FAST-LIO2 — "Where am I?"

Now we reach the first major intelligence block.

For learning purposes, treat FAST-LIO2 as a machine:



The most important output for navigation is:

Odometry

Odometry tells you the estimated:

```
Position:
x
y
z

Orientation:
roll
pitch
yaw

Velocity:
vx
vy
vz
```

For example:

```
Drone pose

x = 4.2 m
y = 2.7 m
z = 1.5 m

roll  = 1°
pitch = -2°
yaw   = 87°
```

Now the drone has an answer to:

> **"Where am I relative to where I started?"**

And this is why GPS isn't necessary.

---

6. There is no magical "GPS replacement"

This is important when explaining the architecture.

Outdoors:

```
GPS
 |
 ↓
global position
```

Inside NIDAR:

```
LiDAR
   +
IMU
   ↓
FAST-LIO2
   ↓
relative pose
```

When the drone starts, conceptually define:

```
START

x = 0
y = 0
z = 0
yaw = 0
```

Then FAST-LIO tracks movement from that reference.

For example:

```
Start
(0,0)
   |
   | 3 m
   ↓
(3,0)
   |
   └─── 2 m ──→
              (3,2)
```

Therefore your entire maze can exist in a **local coordinate system**.

That is enough for NIDAR because the mission asks the system to identify survivor grid coordinates/boxes and display the estimated drone position on the generated map; it doesn't require GPS latitude/longitude. Mission Brief - NIDAR AirMouse

---

7. Point cloud + odometry = map

Now something interesting happens.

At position A:

```
Drone
  X

• • • wall
```

Drone moves.

At position B:

```
        X Drone

• • • wall
```

Because FAST-LIO knows the transformation:

```
Pose A
  ↓
Pose B
```

the new LiDAR measurements can be transformed into the **same map coordinate frame**.

FAST-LIO's mapping step transforms feature points into the global frame using the estimated state and appends them to the existing map. 2010.08196v3

Eventually:

```
Scan 1
   +
Scan 2
   +
Scan 3
   +
Scan 4
```

```
   ↓

3D POINT CLOUD MAP
```

So now we have:

```
MID-360
   |
   ↓
FAST-LIO2
   |
   ├──────────→ ODOMETRY
   |
   └──────────→ POINT CLOUD
```

This distinction is crucial.

Odometry = where the drone is.

Point cloud = what the world looks like geometrically.

---

8. Then why do we need a Map Server?

Because a raw point cloud isn't ideal for planning.

Imagine receiving this:

```
• ••• • ••••
 ••• ••• •
•• ••• • •••
 • ••••• •
```

Humans can visualize it.

A planner wants something simpler:

> Is this location free or occupied?

Therefore:

```
Point Cloud
     |
     ↓
 MAP SERVER
     |
```

The world becomes something conceptually like:

```
██████████████
█..............█
█..............█
█....DRONE.....█
█..............█
██████████████
```

where:

```
█ = occupied
. = free
? = unknown
```

This gives you one of the most important representations in autonomous exploration:

```
FREE
OCCUPIED
UNKNOWN
```

---

9. UNKNOWN is extremely important

At mission start:

```
????????????????
????????????????
??????DRONE??????
????????????????
????????????????
```

Almost everything is unknown.

LiDAR scans.

Now:

```
██████??????????
█....█??????????
█..D.??????????
█....█??????????
██████??????????
```

The drone has discovered some free space and walls.

The boundary between:

```
known FREE space
       and
UNKNOWN space
```

is called a:

Frontier

For example:

```
█████████????????
█........F????????
█........F????????
█...D....F????????
█........F????????
█████████????????
```

`F = frontier`

And now you can understand why FUEL exists.

---

10. FUEL — "Where should I explore next?"

This is **high-level exploration intelligence**.

It doesn't directly tell the motors what to do.

It asks:

> "Which unexplored region should I investigate next?"

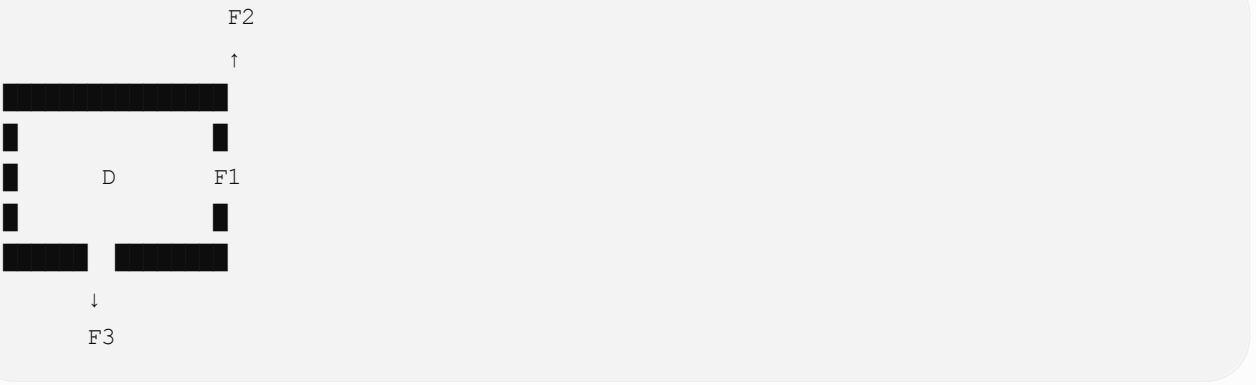
Conceptually:

```
Occupancy Map
     |
     ↓
Find Frontiers
     |
     ↓
Candidate unexplored regions

F1
F2
F3
F4
```

```
      |
      ↓
Evaluate them
      |
      ↓
Choose useful exploration order
      |
      ↓
NEXT VIEWPOINT
```

Suppose:



FUEL may determine that `F3` gives the best exploration value.

So it produces something approximately like:

```
Next viewpoint:

x = 5.4
y = 7.2
z = 1.5
yaw = 90°
```

It answers:

> **WHERE should we go?**

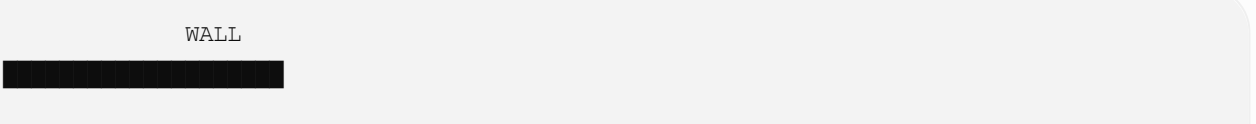
Not necessarily exactly **HOW should the drone physically fly there?**

That is Fast-Planner's responsibility.

---

11. Why ESDF appears before Fast-Planner

Suppose the drone is here:



```
        Drone
          X
```

The planner doesn't merely need:

```
wall exists = yes
```

It benefits from knowing:

```
distance to wall = 1.7 m
```

An ESDF provides a distance-to-nearest-obstacle representation over space.

Conceptually:

```
Wall

0m
|
0.2m
|
0.5m
|
1.0m
|
2.0m
```

Therefore:

```
Occupancy Map
      |
      ↓
     ESDF
      |
      ↓
"How far is this position
 from the nearest obstacle?"
```

That is extremely useful for trajectory optimization.

---

12. Fast-Planner — "How do I reach the viewpoint safely?"

Now FUEL gives:

```
GO HERE

        ★
```

But there might be a wall:

```
Drone                    Goal

  X          ▬▬           ★
             ▬▬
             ▬▬
```

Flying directly:

```
X ──────────────────→ ★
          💥
```

obviously fails.

So Fast-Planner searches for a dynamically feasible route around obstacles.

Conceptually:

```
FUEL
 |
 ↓
Goal / viewpoint
 |
 ↓
FAST-PLANNER ← ESDF
 |
 ↓
Safe trajectory
```

Your diagram calls out two ideas:

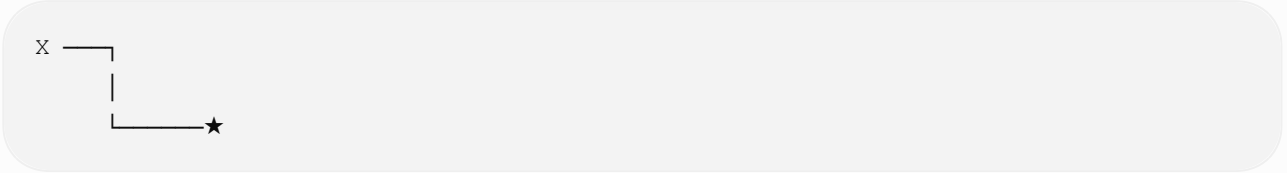
```
Kinodynamic search
       ↓
B-Spline optimization
```

For your team's first mental model:

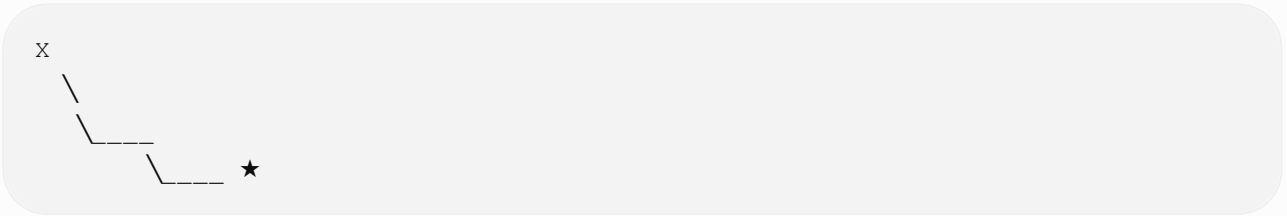
Kinodynamic search = find a feasible rough motion considering drone dynamics.

B-spline optimization = turn/refine that into a smooth, collision-aware trajectory.

So instead of:



you want something closer to:



A quadrotor needs smooth velocity and acceleration commands rather than unrealistic instantaneous direction changes.

---

13. Now MAVROS

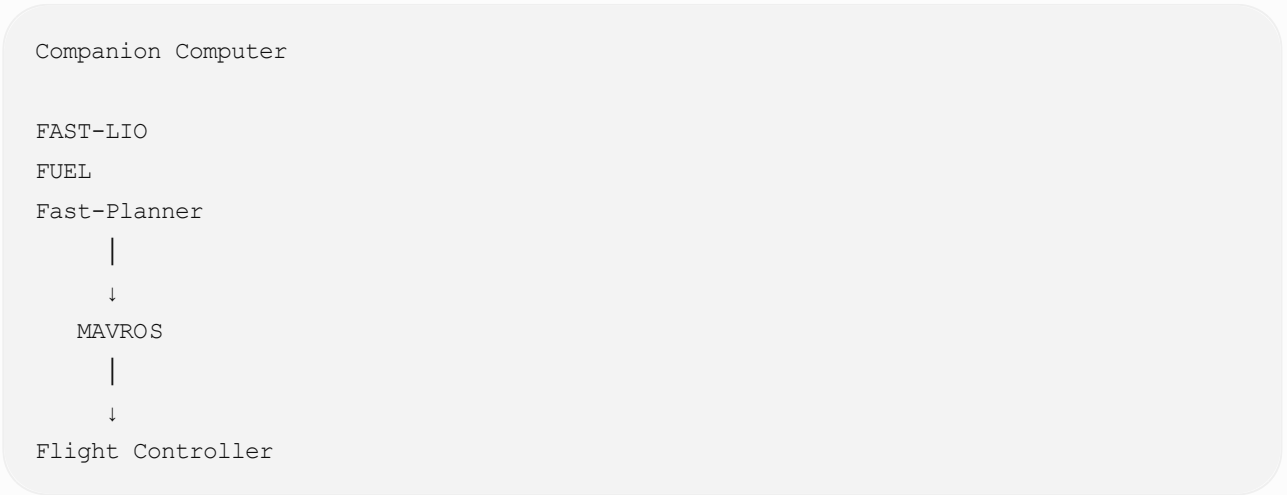
This is another place where architecture diagrams become confusing.

MAVROS isn't the thing deciding:

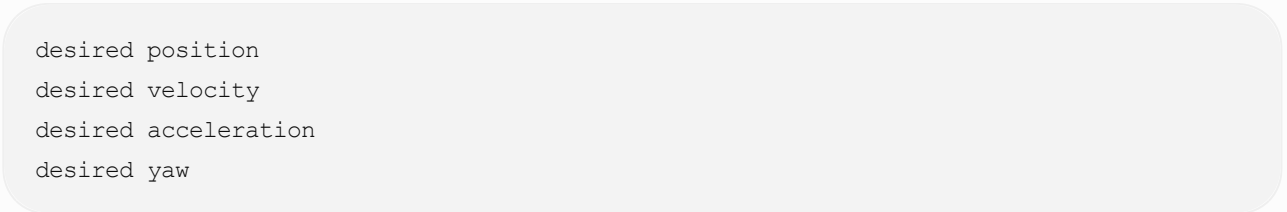
> "Explore room 4."

And it isn't FAST-LIO.

Think of MAVROS as the **communication bridge between autonomy software and the flight controller**.



The planner might produce commands/setpoints corresponding to:



MAVROS transports the relevant information between ROS-side autonomy and the FC/autopilot interface.

---

14. Flight Controller — "How do I make the physical drone obey?"

Suppose Fast-Planner wants:

```
desired position

x = 5.0
y = 3.0
z = 1.5
```

Actual drone:

```
x = 4.7
y = 2.8
z = 1.4
```

The flight controller sees an error.

Conceptually:

```
Desired state
     |
     ↓
   ERROR
     ↑
     |
Actual state
```

Then its control loops determine the thrust/attitude commands needed.

Eventually:

```
Flight Controller
       |
       ↓
      ESC
       |
       ↓
Motor RPM
       |
```

The ESCs are essentially the power electronics controlling the motors.

---

15. Then what is the FC EKF?

You currently have **two state-estimation worlds**:

```
Companion computer:
MID-360 → FAST-LIO2 → LiDAR-inertial odometry

Flight controller:
FC IMU + other aiding → FC EKF
```

This distinction deserves a dedicated discussion later because **how FAST-LIO odometry is supplied to the flight controller and fused/configured is one of the most important integration decisions in the entire project.**

For now, explain it to the team as:

> FAST-LIO provides a strong GPS-denied external pose/odometry estimate; the FC EKF maintains the state required by the autopilot's flight-control system.

Don't think of them as two unrelated systems.

Eventually the architecture needs an intentional relationship between them.

---

16. Now the second parallel pipeline: survivor detection

While all of that is happening:

```
RGB / Thermal Camera
        |
        ↓
    Survivor AI
        |
        ↓
"Person detected"
```

But detection alone isn't enough.

Suppose AI says:

```
PERSON
confidence = 94%
```

NIDAR needs the **survivor's location on the map/grid**, not merely a bounding box in video. The mission brief explicitly requires autonomous identification and tagging of the relevant grid coordinate or grid box. Mission Brief - NIDAR AirMouse

Therefore you need:

```
2D image detection
       |
       ↓
relative localisation
       |
       ↓
3D survivor coordinate
```

For example:

```
Survivor relative to camera:

X = 0.4 m
Y = 3.2 m
Z = -0.3 m
```

But that is still in the **camera coordinate frame**.

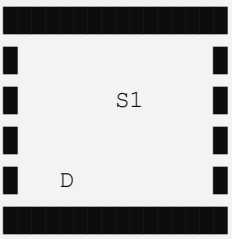
You then need the drone pose from FAST-LIO:

```
Camera survivor coordinate
           +
Drone pose in map
           +
Camera ↔ drone calibration
           ↓
Survivor coordinate in MAP
```

Example:

```
MAP:

```



Now `S1` remains fixed on the map even after the drone flies away.

This is the connection your current ASCII diagram doesn't make visually obvious enough:

```
FAST-LIO odometry ──────┐
                        ↓
Camera → Survivor AI → 3D localisation
                        │
                        ↓
               MAP-FRAME SURVIVOR
```

We'll eventually want to refine that block.

---

17. Optical flow + rangefinder

This is another independent sensing mechanism.

A downward camera observes texture moving across successive images:

```
Frame 1

 •    •
   •
      •

Frame 2

   •    •
     •
       •
```

From that apparent motion, optical flow estimates motion relative to the ground/surface.

A rangefinder provides something like:

```
distance to floor = 1.48 m
```

Together they can provide useful low-altitude/local motion information.

So conceptually:

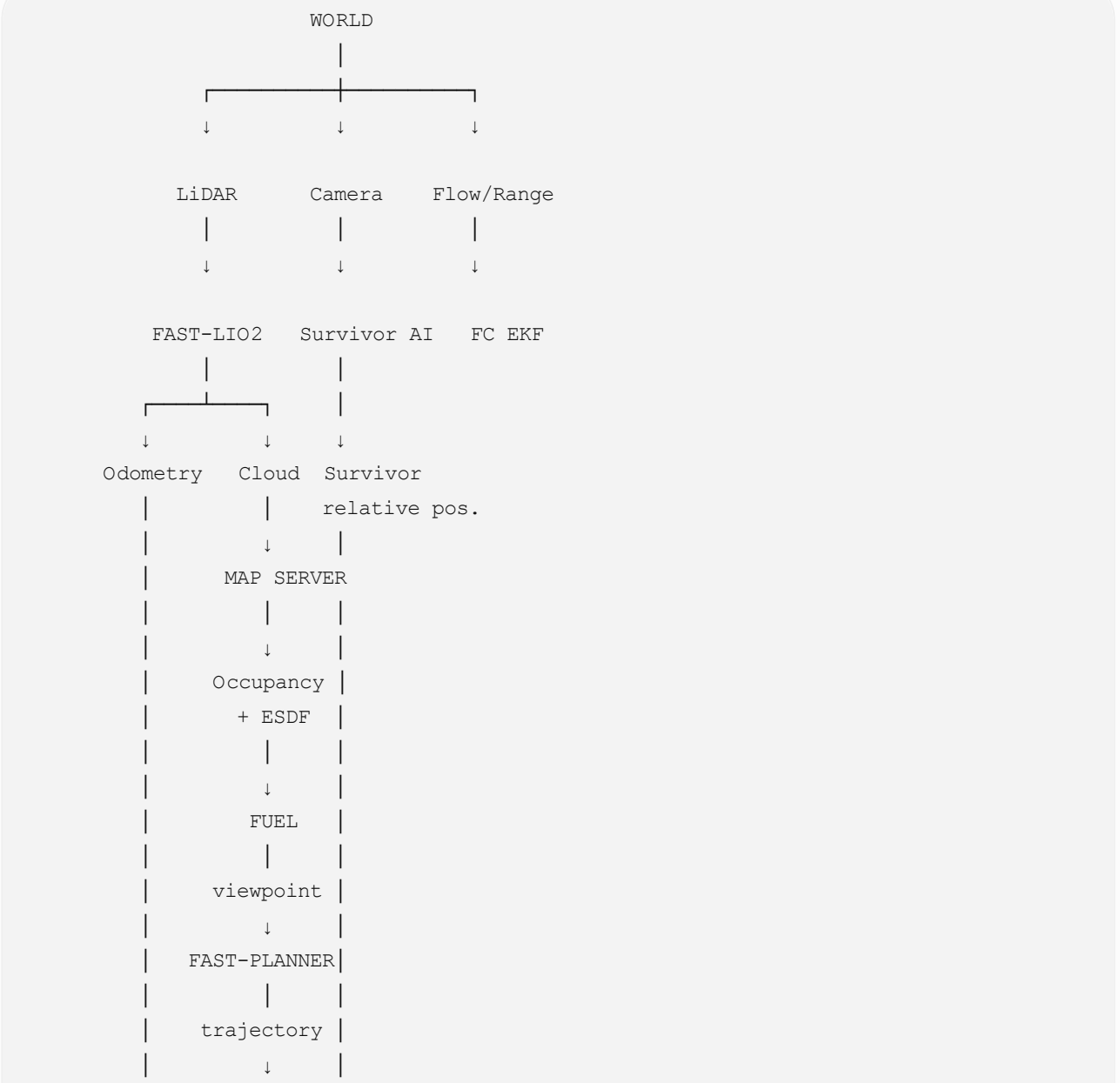
```
Optical Flow
     +
Rangefinder
     ↓
FC EKF
```

This can be particularly useful as another source of aiding rather than making the whole flight stack depend on one estimator.

---

18. Put the whole architecture back together

Now your diagram should make much more sense:



```
          └──────→ MAVROS |
                     |    |
                     ↓    |
               FLIGHT CONTROLLER
                     |
                   ESC
                     |
                  MOTORS
```

And then the drone moves, generating **new sensor measurements**, so the entire cycle repeats.

---

19. The six questions your team should memorize

You can explain almost the complete autonomy stack by asking six questions:

Question	System answering it
What am I seeing geometrically?	MID-360
Where am I?	FAST-LIO2 / state estimation
What does the explored world look like?	Map Server
Where haven't I explored?	FUEL
How can I safely get there?	Fast-Planner
How do I physically fly that trajectory?	Flight Controller

Then add the parallel rescue question:

> **Where is the survivor?**

```
Camera
  ↓
Detection
  ↓
Relative 3D position
  ↓
Transform using drone pose
  ↓
Map coordinate
  ↓
NIDAR grid cell
```

That final transformation is essential for your actual competition objective.

---

20. How I suggest we learn this from here

Don't try learning the entire stack simultaneously. We should move through it in dependency order:

Stage 1 — Coordinate frames and transformations → `LiDAR frame`, `IMU frame`, `body frame`, `camera frame`, `map frame`, `FC frame`. This is the foundation; without it FAST-LIO and survivor localisation become confusing.

Stage 2 — LiDAR + IMU + FAST-LIO2 → what the sensors actually publish, timestamps, point clouds, IMU integration, odometry, drift, motion compensation, and why FAST-LIO works. The FAST-LIO paper's pipeline itself follows LiDAR preprocessing → IMU forward propagation → motion compensation → residual computation → iterated state update → map update. 2010.08196v3

Stage 3 — Mapping → point cloud → voxel/occupancy map → free/occupied/unknown → ESDF.

Stage 4 — Autonomous exploration → frontiers → viewpoints → information gain → FIS → ATSP → FUEL.

Stage 5 — Motion planning → kinodynamic search → B-spline → collision checking → trajectory.

Stage 6 — PX4/ArduPilot integration → FAST-LIO → FC estimator → MAVROS → setpoints → controller → ESC → motors.

Stage 7 — Survivor localisation → camera detection → depth/geometry → camera-frame 3D → body/map transform → grid cell → persistent survivor tag.

Stage 8 — Mission manager → takeoff → explore → survivor bookkeeping → coverage/completion criterion → return to entry/exit → land/mission complete.

The best place to start next is **Stage 1: coordinate frames and transformations**. Once you understand why a point can simultaneously have coordinates in the **LiDAR frame, camera frame, drone/body frame, and map frame**, FAST-LIO, survivor localisation, and the FC integration become dramatically easier to reason about.